Keeping Proof, Execution and Authority Apart When Verifying Coding-Agent Patches: An Empirical Report on a Local-First Prototype

InvariantGuard repository authors*

Workshop draft. Every number is generated from `benchmarks/results/paper_*.json`; see `paper/README.md`.

Abstract

Coding agents need machine-checkable feedback on the patches they write. A single pass/fail or "confidence" signal mixes up different kinds of evidence: a proof under stated assumptions, a finite test, an observed regression, and an inferred property that nobody approved. We describe InvariantGuard, a local-first prototype that checks a patch against an explicit base snapshot with two engines. A static engine handles modeled runtime failures (zero divisors, sequence bounds, `None` attribute access) over a small typed IR for a Python subset. A contract engine replays typed behavioral properties on the base and the patch inside an isolated container. Proposers (templates, a local open-weight model, or the calling coding agent) may suggest contracts, but they never decide whether a contract holds or how much authority it has. Every result keeps proof status, scope, authority, patch relation, coverage and the gate verdict on separate axes. Unsupported code is reported as `UNKNOWN/INCOMPLETE` rather than dropped. We evaluate the implementation on 31 controlled fixtures, on 35 hand-written baseline/patch pairs whose labels were frozen before any tool run, with ablations, with latency on synthetic repositories of about 100 to 10,000 functions, and with three contract proposers on 13 held-out functions. On execution-confirmed seeded labels, the best configuration flagged 18 of 21 introduced defects, abstained on 2 and missed 1, with 0 false positives on 12 negatives. It abstained on 5 of those negatives. The single miss exposed a quick-mode slicing defect in the prototype. Blinded agent-proposed contracts exposed more seeded bugs than the local model (per run: 5, 5, 5 versus 2, 2, 2 of 5). That comparison is confounded by the prompt, the candidate cap, model scale, and model-family overlap with the benchmark author. All benchmarks are synthetic or author-written and ran on one machine. They do not establish effectiveness on natural agent patches.

1 Introduction

Coding agents increasingly propose patches that a person or another agent must accept or reject. Tests, type checkers and a reviewer's reading each give a different kind of evidence. When a tool reports on a patch, the consumer (often another model) needs to know *what* was established, under *which* assumptions and for *which* inputs. It also needs to know whether the claim is about the patch or about code that was already there.

This report describes InvariantGuard, a prototype built to explore that question, and evaluates it empirically. The prototype is deliberately narrow: a Python subset, three static properties, a small typed property language for contracts, and an isolated runner for differential replay. Our contributions are modest and empirical:

- We frame verification results as *evidence axes* that keep proof status, scope validity, authority, patch relation, coverage, run outcome and verdict separate. We use this framing to organize a whole tool (Section 2).
- We built a working, local-first implementation. It combines change-scoped static checking, base-versus-patch contract replay, and contract proposers that can only suggest. The proposers are templates, a local model reached only over loopback, and the calling agent through a context-packet protocol (Sections 3–4).
- We give a reproducible evaluation. It reports explicit denominators, keeps abstentions in every rate, freezes labels before tool runs, and discloses a defect that the evaluation found in the prototype (Section 5).

We do not claim that any component is new. Static runtime-error detection, dynamic invariant discovery, SMT-based checking and agent-driven property-based testing are all established (Section 7).

2 Problem framing: evidence axes

A verification result for a patch answers several independent questions. Collapsing them into one number hides exactly the information that an agent needs to repair a patch, or that a person needs to accept it. InvariantGuard attaches the following axes to every obligation and contract result:

*Disclosure: the prototype, its benchmarks and this report were produced by AI coding agents (Claude Opus 5.5) in a human-directed, specification-driven workflow. The agent-proposed contracts evaluated in Section 5.5 come from Claude Sonnet 5.5, a model of the same family. See Section 6.

axis	values
property result	PROVED_SAFE, PROVED_UNSAFE, POSSIBLY_UNSAFE, TESTED, OBSERVED_FAILURE, OBSERVED_REGRESSION, UNVERIFIED, UNKNOWN
scope validity	EXACT_MODEL, CONDITIONAL, OVERAPPROXIMATE, BOUNDED, EXECUTED
authority	DECLARED, APPROVED, INFERRED, REJECTED, RETIRED
patch relation	INTRODUCED, PRE_EXISTING, RESOLVED, CHANGED, UNDETERMINED
run outcome	COMPLETE, INCOMPLETE, ERROR, CANCELLED
verdict	PASS, WARN, BLOCK, INCOMPLETE, ERROR

The axes encode distinctions that are easy to lose. A satisfiable query over an exact model of the admitted inputs yields PROVED_UNSAFE with a concrete witness. The same query over an overapproximation yields only POSSIBLY_UNSAFE. Python does not enforce type annotations at runtime, so a result that relies on them is CONDITIONAL, not EXACT_MODEL. Passing probes give TESTED, a finite claim, never a proof. A result is an OBSERVED_REGRESSION only when the same admitted input satisfies the property on the base and violates it on the patch, in comparable environments. If both versions fail, the relation is PRE_EXISTING. Authority records who stands behind a property. A contract inferred from old behavior (INFERRED) is not evidence of intent, so it can only warn.

The gate verdict is a policy over the axes, with precedence ERROR > BLOCK > INCOMPLETE > WARN > PASS. Two cases block: an introduced exact failure under a declared domain, and a replayed regression of a declared or approved contract. Conditional or overapproximate failures warn, and so do changes to inferred contracts. Some required obligations can remain UNKNOWN: unsupported constructs, solver timeouts, exhausted budgets and unavailable runners all cause this. They make the run INCOMPLETE, which is never reported as PASS. Required coverage is fixed in the analysis plan before analysis starts.

3 Design

Typed IR and semantic profiles. The Python frontend parses source with the standard `ast` and `symtable` modules. It never imports or executes the repository. Supported module-level functions are lowered to a small typed CFG IR: constants, parameters, integer arithmetic, the three division operators, comparisons, `is None`, `len`, indexing, calls, attributes, branches, ϕ-joins, and an explicit UNKNOWN_EFFECT. Each operation carries a semantic profile rather than a generic meaning [4]. Python `//` and `%` use floor semantics, integers are unbounded, and negative indices are valid in $[-n, n)$. Assertions are lowered with `__debug__` as an explicit runtime-mode input, so behavior under `python -O` is modeled. Some operands are not established exact built-ins, for example a value returned by an unmodeled call, which may dispatch to a user-defined operator. Such operands make the dependent obligation UNKNOWN, or CONDITIONAL when only an annotation supports the type. Loops, nested functions, methods, comprehensions, lambdas, decorators and exception handlers are outside the subset. Their obligations are reported as UNKNOWN with a reason and stay in the denominator.

Static decision procedure. An obligation is a syntactic division, a subscript of an established list or tuple, or an attribute load on a value that may be `None`. The engine first applies unsupported-region checks and reachability. It then tries cheap interval and path facts, and finally, optionally, an SMT query *path condition* $\wedge$ *bad state* in Z3 [5]. UNSAT gives PROVED_SAFE. A SAT model is re-validated by a concrete IR evaluator before it becomes a PROVED_UNSAFE witness. Without the solver, a deterministic bounded witness search remains. A call to a supported function in the same snapshot is instantiated at the call site. Resolving a call across modules makes the result CONDITIONAL on import resolution.

Change-scoped verification. `guard verify --base REF` captures immutable, content-addressed snapshots of two trees: the base (a Git commit or a saved baseline) and the working tree, including uncommitted changes. It aborts or restarts if files change during capture. Symbols are matched across the two snapshots. A dependency graph over calls, imports, global reads and contract scopes seeds an affected slice from the changed symbols. Per-symbol validity keys gate cache reuse; they cover the normalized IR, transitive dependencies, configuration, semantic profile, solver and contract digests. *Quick* mode analyzes the affected slice within a budget (default 2 s) and reports INCOMPLETE if the budget runs out. *Full* mode analyzes the whole scope. Each failure in the candidate is matched to a base obligation, and its witness is replayed against the base model. The result is INTRODUCED, PRE_EXISTING or RESOLVED. When attribution is impossible the relation is UNDETERMINED; attribution is required work, so the run is INCOMPLETE.

Contracts and authority. A behavioral contract is a typed JSON predicate in a small DSL. The DSL has references to arguments, results and before/after argument snapshots; comparisons; boolean connectives; `len`; `sum`; `approx_eq` with mandatory tolerances; `all_between`; `unchanged_argument`; and `repeatability`. There is no indexing and no arithmetic, and candidate strings are never evaluated. Each contract has an input profile (bounded built-in values), a precondition, an authority, and an append-only history of revisions and evidence. With `--execute`, admitted inputs run on *both* snapshots in a Docker container. The container has no network, read-only source, dropped capabilities and resource limits, and the runner checks these from inside before any import. The predicate is evaluated on the host over an exact encoding of built-in values. Counterexamples are shrunk within the domain and confirmed in fresh runs. An intentional API change is accepted through an auditable contract revision, never by deleting evidence.

Proposers, not judges. Contract discovery has five sources. All of them share one parser and one validation pipeline: typed

DSL, symbols, budgets, an admissible precondition, a behavioral non-vacuity filter, de-duplication, and optional isolated falsification. Every accepted candidate is recorded as `INFERRED` at most.

- *Templates*: purity, length preservation, assertions and repeatability.
- *A local open-weight model*: reached only over a validated loopback endpoint, pinned by digest, never downloaded during a scan.
- *Bring-your-own-key adapters* for OpenAI-compatible and Anthropic endpoints: opt-in, with https and explicit permission required for any non-loopback host. Each data egress is recorded per function. There is no fallback in either direction.
- *The calling coding agent*: it reads a *context packet* (MCP `get_context_packet` or `guard contracts context`) and submits a batch bound to the packet's digest. A batch for a different packet is rejected as stale. The batch schema has no field for authority, approval or evidence, and the proposer's identity is taken from the transport and labeled as self-reported.

The context packet is AST-only and budgeted (12,000 bytes), and it frames all source as untrusted data. It is the same object for the local model and for the agent. Model and agent output never becomes a proof assumption, and no tool can approve a contract.

Agent interface. All commands emit schema-validated JSON with stable finding identifiers, witnesses, source spans and explicit coverage. A stdio Model Context Protocol server [2, 3] exposes verification, evidence retrieval and contract proposal over the same orchestration code as the CLI. It has no tool that approves contracts, changes baselines or executes commands.

4 Implementation

The prototype is 17,617 lines of Python in 68 files (engine 0.5.0, semantics profile `python3.13-cpython-builtin-v4`). The core uses only the standard library. Z3 5.1.0 is an optional solver dependency, and the MCP SDK is an optional extra. On the evaluation host the test suite (8,404 lines) reported *788 passed, 2 skipped, 1 xfailed* in 206 s. The skips are opt-in tests against the real model, and the expected failure records the defect discussed in Section 5.4. Contract checks use Docker 29.5.2 with a pinned `python:3.13-slim` image. The local model is `qwen2.5-coder:7b` (Q4_K_M, pinned by digest) [7], served by Ollama 0.40.1. Dependencies are pinned in a lock file that equals the environment's `pip freeze`.

5 Evaluation

Setup. All runs used one machine: Mac16,12 (Apple M4, 16 GB RAM), macOS 15.6 (24G84), CPython 3.13.9, on 2026-10-08, 2026-10-09 (UTC). The host also ran unrelated applications. Among them were containers of another project, which Docker Desktop restarted when it was launched for our runner. Sampled once a minute from 2026-10-08 23:43 to 2026-10-09 03:26 UTC (208 samples), the 1-minute load average had a median of 7 and a maximum of 137 on 10 logical CPUs, and up to 18.5 GB of swap was in use. Timings are therefore noisy (see RQ4). `benchmarks/run_all.py` produces every number below, and `paper/make_tables.py` typesets them. We address the research questions of the project's specification (`docs/12` in the repository) only as far as the data allows.

- **RQ1**: How faithful is the static engine on controlled semantics, and how often does it abstain?
- **RQ2**: How does change-aware verification behave on seeded patches, including safe, pre-existing and intentional changes? Does differential evidence avoid false blocks?
- **RQ3**: How often does combining static and dynamic checking improve actionable findings?
- **RQ4**: Does conservative change-aware (quick) verification agree with full verification, and what does it cost?
- **RQ5**: Do mechanically checked proposer candidates (templates, a local model, a coding agent) add properties that expose bugs?

The last specified question, "what does honest abstention cost?", runs through all five.

Ground truth. Labels never come from the tool. For a *defect* label, we executed a hand-written witness (for patches, on both versions) in a separate `python -I` process and observed the failure. For a *safe* label, we wrote a hand argument and corroborated it, without proving it, on a finite input grid. Cases whose truth depends on code outside the fixture are unlabeled and excluded from precision and recall, but they are still counted. Abstentions (`UNKNOWN`, `INCOMPLETE`) remain in all denominators. We report recall two ways: with abstentions counted as misses (Rec._a), and over decided cases only (Rec._d).

5.1 RQ1: controlled semantic fixtures

The 31 controlled fixtures exercise one obligation each. They cover guarded and unguarded division, infeasible branches, relational guards, annotation-only domains, floor semantics, assertion runtime mode, empty lists, negative indices, `None` flow and a changed callee. They also cover unsupported constructs: loops, methods, decorators, closures, unknown calls, floats, `isinstance` with `bool`, and a nonlinear query whose smallest known solution has 16-digit coordinates. 28 cases have an independent label (18 defects, 10 safe); 3 do not. All 31 results matched the specified expectation, with and without the solver (Table 1).

With the solver, the engine raised no false alarm (0 FP) and missed no defect that it decided (0 FN). However, it abstained on 8 of the 18 defects and on 1 safe case, so Rec._a is 0.56 while Rec._d is 1.00. The abstentions are the documented subset boundary (loops, methods, decorators, closures, floats, `isinstance`, list element values, aliasing, unknown callees) plus a solver timeout. Disabling the solver changed 2 result(s): two-variable-equality, unknown-call-unrelated. Intervals cannot prove the relational guard $a \neq b \Rightarrow a - b \neq 0$, so that obligation becomes `UNKNOWN`. The `POSSIBLY_UNSAFE` result on an overapproximated

path also becomes UNKNOWN. These fixtures were written during development and serve as regression tests. They therefore measure conformance to the specification, not generalization.

5.2 RQ2: seeded baseline/patch pairs

We wrote 35 small baseline/patch pairs for this evaluation. They cover nine classes:

- removed, weakened or reordered division guards;
- changed callees and constants that make unchanged callers unsafe;
- off-by-one indices and removed emptiness guards;
- None introduced directly or through a callee;
- contract regressions (normalization, purity, length, range);
- intentional changes, with and without a contract revision;
- safe refactors;
- cases outside the supported subset;
- pre-existing failures.

Labels, oracle inputs and our a-priori expectation of the tool's outcome were frozen at 2026-10-08T17:44:28Z (manifest digest 94f276b0810d...), before any tool run on these cases. Execution disagreed with 0 labels. Each case runs in a throwaway Git repository through the real CLI, with the patch left as uncommitted changes. Table 2 gives the counts per class; the appendix lists every case.

Over the 21 defect and 12 non-defect labels (Table 3), static quick verification flagged 13 defects, abstained on 4 and cleared 4, with 0 false positives. Three of the cleared defects are contract regressions that have no static obligation at all. The static engine is *correct* about its own properties there, but it cannot see the behavior the label is about. The best configuration adds contract replay (--execute) for cases with contracts. It flagged 18 of 21 (Rec.$_a$ 0.86) at precision 1.00. It still abstained on U01, U04, which are a loop and an unmodeled math.gcd. Its one false negative, U02, is a guard removed inside a nested function: quick mode reports PASS where full mode reports INCOMPLETE (Section 5.4).

Two flagged defects (S04, S07) produced WARN rather than BLOCK. Their witnesses are CONDITIONAL: one rests on annotations only, and the other on import resolution. This is the intended policy for claims that do not hold unconditionally.

Safe, pre-existing and intentional changes. No non-defect case was blocked (0 blocks on negatives). The cost is abstention: 5 of the 12 negatives (I01, N06, U03, P01, P02) ended INCOMPLETE, for three reasons:

- Required static obligations sit inside a comprehension or a loop, which the subset cannot decide (I01, N06, U03, P02). This happens even when the contract replay was TESTED.
- In P02 the replay correctly reported the contract failure as OBSERVED_FAILURE PRE_EXISTING.
- In P01 a pre-existing division failure moved into a temporary variable, and the engine could not attribute the witness across the edit (UNDETERMINED). Attribution is required work, so the run is INCOMPLETE, not PASS.

The four cases written as intentional changes have two labels: I02 and I04 are labeled *intentional change*, and I01 and I03 are labeled *no defect*: I01 revises its declared contract together with the change, and I03 raises a documented ValueError while its division stays guarded. Of the four, the tool blocked I04 and did not block I01, I02, I03. I04 changes behavior against a *declared* contract without revising it. The policy requires an explicit, audited revision, and the revised variant (I01) did not block. I02 contradicts only a template-inferred purity contract. That produced an OBSERVED_REGRESSION with a WARN verdict, as specified for inferred authority. Whether the I04 block is a "false block" depends on whether the unrevised contract still states the maintainer's intent, so we report it rather than score it.

5.3 RQ3: static and dynamic evidence

On the 11 seeded cases that carry contracts, Table 4 separates the engines. Static analysis alone flagged 0 cases and abstained on 5. Most contract regressions involve no modeled runtime failure, and these functions use comprehensions or unmodeled builtins. Contract replay alone flagged 5 with 0 false positives and abstained on 0. When both ran in one execution, the regressions still blocked: BLOCK outranks INCOMPLETE. However, the verdict inherits the static engine's missing coverage. 3 non-defect cases ended INCOMPLETE even though contract replay found no regression: I01 and N06 were TESTED, and the P02 failure was correctly attributed as PRE_EXISTING. This is the intended conservative behavior, but it shows what honest abstention costs when the static subset is narrow. Dynamic checking is also expensive. The median --execute run took 4,639 ms of wall time, against 408 ms for static quick runs; the runner capability probe alone took 955 ms.

5.4 RQ4: incremental versus full verification

Seeded cases. Cold quick and full runs agreed on all 30 obligations they shared (0 mismatches). So did warm-cache quick runs (30 compared, 0 mismatches). Disabling the solver changed 0 of 30 seeded obligations. The deterministic bounded witness search found every seeded witness, including the nonlinear one in S02. The verdicts nevertheless differed in 1 case (U02): removing a guard *inside a nested function* yields PASS in quick mode and INCOMPLETE in full mode. The cause is in the prototype. The change fingerprint of an unsupported symbol hashes only its division nodes, so an edit elsewhere in its body does not mark it as modified. Quick mode then leaves the required UNKNOWN obligation outside the slice. Comparing obligations alone would have hidden this; it surfaced only because the benchmark compares labels and verdicts. The reported numbers come from the unmodified engine, and an expected-failure test records the defect.

Synthetic repositories. The generated projects have modules of 25 functions that import a shared core module. We test three changes:

- *leaf*: one function body changes;
- *hub*: a constant read by most functions changes;
- *callee*: a shared helper changes so that callers' divisors can become zero. This introduces real defects in unchanged

Table 1: Controlled semantic fixtures (one obligation per case). Result counts over all cases; "=spec" is agreement with the specified expected result; the confusion counts use only cases with an independent label (defect: witness executed; safe: hand argument plus finite grid). Rec._{a} counts abstentions as misses; Rec._{d} uses decided cases only.

	SAFE	UNSAFE	POSS.	UNK.	=spec	TP	FP	FN	TN	Abst.	Prec.	Rec._{a}	Rec._{d}	Abst. rate
solver on	9	10	1	11	31/31	10	0	0	9	9	1.00	0.56	1.00	0.32
solver off	8	10	0	13	31/31	10	0	0	8	10	1.00	0.56	1.00	0.36

Table 2: Seeded patches by class. Left block: static quick verification (`guard verify --mode quick`); right block: the best available configuration (the `--execute` run for cases with contracts, otherwise static quick). A = abstained (required coverage missing: verdict INCOMPLETE without a flagged finding), kept in every denominator. The two *intentional* labels are excluded from the counts and discussed in the text. Labels confirmed by executing both versions in a separate interpreter.

			static quick					best available				
class	n	labels	TP	FN	A	FP	TN	TP	FN	A	FP	TN
division by zero	6	6 defect	6	0	0	0	0	6	0	0	0	0
changed callee	2	2 defect	2	0	0	0	0	2	0	0	0	0
index bounds	3	3 defect	3	0	0	0	0	3	0	0	0	0
None flow	2	2 defect	2	0	0	0	0	2	0	0	0	0
contract regression	5	5 defect	0	3	2	0	0	5	0	0	0	0
unsupported construct	4	3 defect, 1 none	0	1	3	0	0	0	1	3	0	0
safe refactor	7	7 none	0	0	1	0	6	0	0	1	0	6
intentional change	4	2 intent., 2 none	0	0	1	0	1	0	0	1	0	1
pre-existing failure	2	2 none	0	0	2	0	0	0	0	2	0	0
all	35	21 defect, 12 none, 2 intent.	13	4	9	0	7	18	1	7	0	7

callers.

Quick and full runs agreed on every shared obligation where both made a claim (13,210 compared, 0 mismatches; Table 5). The first run also had 0 mismatches in 13,210 comparisons. When the quick budget ran out, quick mode reported 5,305 obligations as `UNKNOWN` that full mode decided. In those runs 2,031 of the 2,164 introduced findings of the full runs did not appear in the quick report. Each of those quick runs ended `INCOMPLETE`, never `PASS`. One verdict pair differs the other way: for the leaf change at about 10,000 functions, quick mode said `PASS` while the compared cold full run was `INCOMPLETE`. The full run left obligations outside the slice unresolved (see Latency below), and quick mode by design states that it did not re-verify the unaffected symbols. Full mode reported the callee defects in unchanged callers in other modules with a `WARN` verdict, not `BLOCK`: a cross-module result is `CONDITIONAL` on import resolution.

Latency. Latency on this host was sensitive to load spikes from other processes, so we measured it twice and fixed in advance which run to report (DECISIONS I-111 in the repository; Appendix Table 10). The first run, in pipeline order, had a median 1-minute load of 7.2 on 10 logical CPUs, with spikes up to 93.7, and 8.9–11.9 GB of swap in use (sampled from mid-run on). We report the repeat at the end of the pipeline, which had a median load of 3.9 (range 2.0–10.4) and 11.9–15.4 GB of swap. Per condition, the first run's p50 wall time was 0.87–4.16 times the repeat's (median 1.07).

In the first run, 8 of 263 full-mode runs ended `INCOMPLETE`; in the repeat, 2 of 263 did. None of them exhausted the 600 s full budget. Each left 1–5 required obligations `UNKNOWN` out of more than a thousand. That is consistent with per-query solver timeouts under CPU contention, but the benchmark did not store per-run reasons, so we have not verified the cause. The same configuration can therefore give `PASS` or `INCOMPLETE` depending on host load. In the reports we compared, the effect was missing coverage, never a contradictory claim.

Figure 1 and Table 8 give wall time per `guard verify` process. There were $n = 30$ runs per condition at about 100 and 1,000 functions. At about 10,000 functions we used $n = 5$, because a single cold full run takes minutes; this falls short of 30 repeats.

- *One-function change, quick mode.* 493 ms at p50 (p95 517) at about 100 functions; 1,445 ms (p95 1,488) at about 1,000; 14,335 ms at about 10,000.
- *Warm caches.* They help full mode much more than quick mode. At about 1,000 functions, full mode took 9,334 ms cold versus 3,388 ms warm, while quick mode took 1,445 ms cold versus 1,435 ms warm. Quick mode is dominated by capturing and lowering *both* snapshots completely in order to compute validity keys (Table 9).
- *Hub change, quick mode.* It ended INCOMPLETE cold and PASS warm at about 1,000 functions, and INCOMPLETE cold and INCOMPLETE warm at about 10,000.
- *Memory.* Peak RSS was at most 756 MB.

Our specification sets a provisional target of two seconds p95 for quick static checks. In this run it was met at about 100 functions for both changes and at about 1,000 functions for the leaf change. It was not met for the hub change at 1,000 functions (cold p95 4,711 ms, warm p95 2,741 ms) or for any

Table 3: Seeded patches: defect detection over execution-confirmed labels (intentional changes excluded; see text). A = abstentions, kept in the denominators; Rec._a counts them as misses. The last row uses the *execute* run for cases with contracts and the static quick run otherwise.

configuration	pos	neg	TP	FN	A_{pos}	FP	TN	A_{neg}	Prec.	Rec._a	Rec._d	Abst.
static quick	21	12	13	4	4	0	7	5	1.00	0.62	0.76	0.27
static quick, no solver	21	12	13	4	4	0	7	5	1.00	0.62	0.76	0.27
static full	21	12	13	3	5	0	7	5	1.00	0.62	0.81	0.30
static quick, warm cache	21	12	13	4	4	0	7	5	1.00	0.62	0.76	0.27
static + dynamic (execute)	21	12	18	1	2	0	7	5	1.00	0.86	0.95	0.21

Table 4: Static-only, dynamic-only and combined verification on the 11 seeded cases that carry contracts (labels confirmed by execution; intentional changes excluded). *Static only*: the quick run without `--execute`; *dynamic only*: the contract results of the `--execute` run; *combined*: that run's verdict.

engines	cases	TP	FN	A_{pos}	FP	TN	A_{neg}
static only	9	0	3	2	0	1	3
dynamic only	9	5	0	0	0	4	0
combined	9	5	0	0	0	1	3

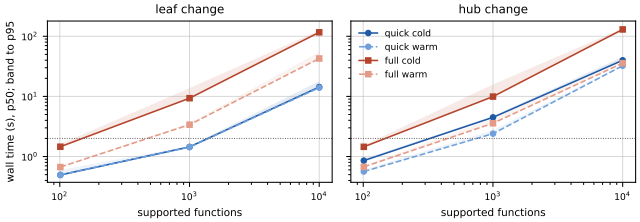


Figure 1: Wall time of `guard verify` versus repository size (log-log; line = p50, band = p50 to p95). Dotted line: the 2 s default quick budget.

change at 10,000 functions. We report the measurement; we do not restate the target as achieved.

5.5 RQ5: contract proposers

We compare three proposers on 13 held-out functions. 5 of them contain a seeded bug that contradicts the function's own docstring; 5 of these bugs are expressible in the DSL. Every candidate passed through the same validation, non-vacuity, de-duplication and Docker falsification (32 probes). A refuted claim was then checked against reference behavior: the intended reference implementation for seeded functions, the fixture itself otherwise. It was checked at the counterexample and at 32 seeded inputs, which is a mechanical check, not a developer's review. The proposers:

- *Templates*: deterministic.
- *Local model*: `qwen2.5-coder:7b`, temperature 0, fixed seed, at most three candidates per function, 3 repeats.
- *Agent-proposed*: 3 independent Claude Sonnet 5.5 subagents. Each saw only the 13 context packets (identical to the model's packets) and the protocol README, and wrote up to eight candidates per function. The session transcripts show that no subagent read any other file or ran the tool. Scoring submitted the batches through the same code path as `guard contracts propose`, with templates in the same pass. Table 6 shows the results.
- *Templates* produced 23, 23, 23 candidates per repeat, all `TESTED`, and exposed 0, 0, 0 seeded bugs. They describe shape (purity, length, repeatability), not intent.
- *The local model* produced 35, 35, 35 raw candidates and exposed 2, 2, 2 seeded bugs per repeat (clip_negatives, dedupe_sorted). 8, 6, 8 of its claims per repeat were false about the intended behavior; they were refuted, so none would have been recorded. Its answers differed across repeats at temperature 0 with a fixed seed (3 distinct response sets in 3 repeats).
- *The agent* produced 81, 89, 84 candidates per run, all schema-valid and non-vacuous, with no batch rejected. It exposed 5, 5, 5 of the 5 seeded bugs per run (clamp_percent, clip_negatives, count_positive, dedupe_sorted, normalize_to_unit), with 0, 0, 0 false claims refuted per run. Bugs exposed in every agent run: clamp_percent, clip_negatives, count_positive, dedupe_sorted, normalize_to_unit. Bugs exposed in every local-model repeat: clip_negatives, dedupe_sorted. Bugs that only the agent exposed: clamp_percent, count_positive, normalize_to_unit.

This comparison is *not* controlled, and four differences favor the agent:

- *Different instructions.* The local model's system prompt asks it to "claim only what the code or its documentation supports" and to propose fewer candidates rather than guess. The agent harness additionally asked for "properties that a maintainer would plausibly intend, including ones that might expose bugs". The README the agents read describes the seeded-bug metric.
- *Different caps*: eight candidates per function versus three.
- *Model scale.* The agent is a much larger hosted model than a 7B quantized model running on the laptop.
- *Same family as the author.* The held-out functions, their docstrings and the DSL were written by a model of the same family (Section 6).

The result therefore shows that the pipeline accepts agent proposals and that the same falsification filter applies to them. It does not show that one proposer is better in general.

Cost. Model inference dominated the local arm: per function, 92.2 s at p50 and 128.4 s at p95, over 39 answered calls; the inference server peaked at 4,918 MB RSS. Discovery is therefore

Table 5: Incremental (quick, affected slice) versus full analysis at equal configuration: obligations of the quick report compared by ID with the full report. *mismatch*: both make a claim and the claims differ; *cov. loss*: quick is UNKNOWN (budget) where full decides; *missed*: INTRODUCED findings of the full run absent from the quick run. *callee*: a shared helper change that introduces real defects in unchanged callers.

fns	change	compared	mismatch	cov. loss	intro.$_{\text{full}}$	intro.$_{\text{quick}}$	missed	verdict q/f	warm c/m/l
101	leaf	3	0	0	0	0	0	PASS/PASS	3/0/0
101	hub	61	0	0	0	0	0	PASS/PASS	61/0/0
101	callee	48	0	0	21	21	0	WARN/WARN	–
1,001	leaf	3	0	0	0	0	0	PASS/PASS	3/0/0
1,001	hub	722	0	64	0	0	0	INCOMPLETE/PASS	722/0/0
1,001	callee	517	0	161	215	112	103	INCOMPLETE/WARN	–
10,001	leaf	3	0	0	0	0	0	PASS/INCOMPLETE	3/0/0
10,001	hub	7,132	0	2,001	0	0	0	INCOMPLETE/PASS	7,132/0/2,001
10,001	callee	4,721	0	3,079	1,928	0	1,928	INCOMPLETE/WARN	–

Table 6: Contract proposers on the held-out set (13 functions, 5 seeded bugs), all through the same validation, non-vacuity filter, de-duplication and Docker falsification (32 probes). *Templates*: deterministic, 3 repeats. *Local model*: qwen2.5-coder:7b, temperature 0, at most 3 candidates per function, 3 repeats. *Agent-proposed*: Claude Sonnet 5.5 subagents that saw only the context packets and the protocol README, at most 8 candidates per function, 3 independent runs; templates ran in the same pass so duplicates of template claims are excluded from *new beyond templates*. Rates use each source's raw candidates as denominator. Wall time includes model inference (local model) or only validation and falsification (agent: proposals were written beforehand; each run took under two minutes by its session timestamps).

metric (one value per repeat)	templates	local model	agent-proposed
raw candidates	23, 23, 23	35, 35, 35	81, 89, 84
schema-valid rate	1.00, 1.00, 1.00	0.77, 0.77, 0.83	1.00, 1.00, 1.00
non-vacuous rate	1.00, 1.00, 1.00	0.66, 0.63, 0.69	1.00, 1.00, 1.00
mech. checkable rate	1.00, 1.00, 1.00	0.66, 0.63, 0.69	1.00, 1.00, 1.00
distinct validated	23, 23, 23	11, 9, 12	58, 66, 65
new beyond templates	–	11, 9, 12	58, 66, 65
TESTED (distinct)	23, 23, 23	1, 1, 2	52, 56, 56
REFUTED (distinct)	0, 0, 0	10, 8, 10	6, 10, 9
false claim	0, 0, 0	8, 6, 8	0, 0, 0
valid for intent	0, 0, 0	2, 2, 2	6, 10, 9
seeded bugs exposed (of 5)	0, 0, 0	2, 2, 2	5, 5, 5
wall time (s)	21, 21, 23	1071, 1398, 1174	85, 99, 83

an explicit command and never part of `guard verify`. By the session transcripts' timestamps, each agent run wrote its proposals in under two minutes; agent token usage was not metered. Validating and falsifying them took 1.5 min per run.

Transport check. We also pointed the bring-your-own-key OpenAI-compatible adapter at the *same* local Ollama server's `/v1` endpoint for one pass over the 13 functions. This checks transport, not quality. The adapter reported AVAILABLE. All 13 of 13 functions were answered, with structured output json_schema (13) and served model qwen2.5-coder:7b. The egress record states that context left the machine: no. The pass produced 35 raw candidates at a schema-valid rate of 0.83, and 3 seeded bugs were exposed, with a per-function latency of 73.4 s at p50. We do not treat these as a second quality estimate, for three reasons: it is the same model; it is a single pass; and the request differs from the native arm (no digest pin, no explicit context size, `response_format` instead of Ollama's `format`). No hosted third-party provider was evaluated. The Anthropic adapter and hosted OpenAI-compatible endpoints have been exercised only against local fakes in the test suite.

6 Threats to validity

- *Author-written benchmarks, and an AI author.* The fixtures, the seeded patches, the held-out functions, the ground-truth oracle and the tool were all written by AI coding agents (Claude Opus 5.5) working from one specification. Those agents knew the supported subset. Labels were frozen before the tool ran, but case selection still reflects that knowledge. The seeded set is small (35 cases) and is not a sample of natural agent mistakes.
- *Same model family as the proposer.* The agent proposer (Claude Sonnet 5.5) comes from the same family as the model that wrote the held-out functions and their docstrings. It may share their idioms. It was blinded to everything except the context packets and the protocol README, but that README states that seeded bugs are scored. The prompts and candidate caps also differ between arms (Section 5.5).
- *Not held out.* The semantic fixtures were used as regression

tests during development. One ground-truth field (the source line that a safe label refers to, for a function with two division sites) was added after a smoke run. No label changed.

- *One local model, one machine.* We evaluated one 7B quantized open-weight model on one laptop-class host under uncontrolled background load. Latency and model results may not transfer.
- *Ground truth.* Defect labels are confirmed by execution. Safe labels rest on hand arguments plus finite grids, which corroborate but do not prove. The check of whether a model claim is valid uses our own reference implementations and DSL evaluator.
- *Small counts.* Precision and recall over a few dozen cases have wide uncertainty. We report raw counts so that readers can compute their own intervals, and we report no per-finding probabilities.

7 Related work

REDO [8] detects runtime errors in coding-agent patches without executing them, by combining static analysis with an LLM. That is close to the goal of our static engine. We have not run REDO, and our benchmark is not comparable to its benchmark. In agentic property-based testing [9, 10], an agent infers properties from code and documentation, tests them with Hypothesis [11], and reports bugs. Our contract discovery overlaps with this work. Our version restricts every proposer, including a coding agent, to a typed DSL whose candidates are falsified mechanically and carry no authority. Daikon [6] and DIG [1, 12] infer likely invariants from executions. Their observation that inferred invariants describe observed behavior rather than intent motivates our separate authority axis. Z3 [5] is our solver, and agents consume our results over MCP [2]. We make no comparative performance claims against any of these systems.

8 Limitations

- *Narrow static subset.* It excludes loops, comprehensions, methods, closures, exception handlers and floating-point values. On realistic code many required obligations will be `UNKNOWN`, so honest verdicts will often be `INCOMPLETE`.
- *Quick mode.* It recomputes both snapshots in full and misses the latency target at scale. Its fingerprint defect (RQ4) can yield `PASS` for a change inside an unsupported nested function.
- *Finite contract evidence.* Contract checks give finite evidence over bounded built-in inputs only.
- *Coverage.* Only Python is implemented. The remote adapters have not been run against a hosted provider.
- *Agent repair.* The repository has one recorded agent repair loop. It is a single demonstration, not a rate, and its agent was a local-model harness, not a production coding agent.

9 Conclusion

Keeping proof, scope, authority, patch relation and coverage apart made the prototype's results checkable. In our seeded set, every flagged defect carried either a validated witness or a replayed counterexample. Abstentions were counted instead of hidden, and the same discipline exposed a real defect in the tool itself. Treating every proposer (templates, a local model or a coding agent) as an untrusted source of typed, falsifiable candidates let three very different proposers share one pipeline, with no candidate gaining authority. The evidence here is engineering evidence on small synthetic benchmarks. Several questions remain open: effectiveness on natural agent patches, the value of proposed contracts to developers, and performance at repository scale. Answering them requires held-out real-world tasks, multiple independent models and authors, and human review.

References

[1] DIG: Numerical invariant generation (source repository). `https://github.com/dynaroars/dig`. Accessed 2026-10-08.

[2] Model Context Protocol specification, revision 2025-06-18: Transports. `https://modelcontextprotocol.io/specification/2025-06-18/basic/transports`, . Accessed 2026-10-08.

[3] Model Context Protocol Python SDK. `https://github.com/modelcontextprotocol/python-sdk`, . Version 2.3.0 used in this work.

[4] The Python language reference: Expressions. `https://docs.python.org/3/reference/expressions.html`. Accessed 2026-10-08.

[5] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, Lecture Notes in Computer Science, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.

[6] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, December 2007.

[7] Binyuan Hui, Jian Yang, Zeyu Cui, et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024. URL `https://arxiv.org/abs/2409.12186`.

[8] Shou Li, Andrey Kan, Laurent Callot, Bhavana Bhasker, Muhammad Shihab Rashid, and Timothy B. Esler. REDO: Execution-free runtime error detection for COding agents. arXiv:2410.09117 (v1, 10 October 2024), 2024. URL `https://arxiv.org/abs/2410.09117`.

[9] Muhammad Maaz, Liam DeVoe, Zac Hatfield-Dodds, and Nicholas Carlini. Agentic property-based testing: Finding bugs across the Python ecosystem. arXiv:2510.09907; 4th Deep Learning for Code Workshop at NeurIPS 2025, 2025. URL `https://arxiv.org/abs/2510.09907`.

[10] Muhammad Maaz, Liam DeVoe, Zac Hatfield-Dodds, and Nicholas Carlini. Finding bugs across the Python ecosystem with Claude and property-based testing. Anthropic research post, 14 January 2026, 2026. URL `https://www.anthropic.com/research/property-based-testing`.

[11] David R. MacIver, Zac Hatfield-Dodds, and Many Other Contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.

[12] ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. DIG: A dynamic invariant generator for polynomial and array invariants. *ACM Transactions on Software Engineering and Methodology*, 23(4):1–30, 2014. doi: 10.1145/2556782.

A Per-case and latency details

The tables below are generated from the same results files as the main text.

Table 7: Seeded baseline/patch pairs, one row per case (labels frozen before any tool run; "?" = execution did not confirm the label). Predictions: F = flagged (an INTRODUCED/CHANGED failure finding), C = clear, A = abstained (required coverage missing); the verdict follows the slash. *execute* adds dynamic contract checks in the Docker runner (cases with contracts only).

ID	class	label	quick	no-solver	full	execute
S01	div. guard removed	defect	F/BLOCK	F	F/BLOCK	–
S02	div. expression changed	defect	F/BLOCK	F	F/BLOCK	–
S03	div. guard weakened	defect	F/BLOCK	F	F/BLOCK	–
S04	div. guard weakened	defect	F/WARN	F	F/WARN	–
S05	div. guard reordered	defect	F/BLOCK	F	F/BLOCK	–
S06	changed callee	defect	F/BLOCK	F	F/BLOCK	–
S07	changed callee	defect	F/WARN	F	F/WARN	–
S08	changed constant	defect	F/BLOCK	F	F/BLOCK	–
S09	index guard removed	defect	F/BLOCK	F	F/BLOCK	–
S10	index off by one	defect	F/BLOCK	F	F/BLOCK	–
S11	index off by one	defect	F/BLOCK	F	F/BLOCK	–
S12	None guard removed	defect	F/BLOCK	F	F/BLOCK	–
S13	None from callee	defect	F/BLOCK	F	F/BLOCK	–
D01	contract: normalization	defect	A/INCOMPLETE	A	A/INCOMPLETE	F/BLOCK
D02	contract: purity	defect	C/PASS	C	C/PASS	F/BLOCK
D03	contract: length	defect	C/PASS	C	C/PASS	F/BLOCK
D04	contract: range	defect	C/PASS	C	C/PASS	F/BLOCK
D05	contract + static div.	defect	A/INCOMPLETE	A	A/INCOMPLETE	F/BLOCK
I01	intentional, revised	none	A/INCOMPLETE	A	A/INCOMPLETE	A/INCOMPLETE
I02	intentional, inferred	intent.	C/PASS	C	C/PASS	F/WARN
I03	intentional, static	none	C/PASS	C	C/PASS	–
I04	intentional, unrevised	intent.	A/INCOMPLETE	A	A/INCOMPLETE	F/BLOCK
N01	safe refactor	none	C/PASS	C	C/PASS	–
N02	safe refactor	none	C/PASS	C	C/PASS	–
N03	safe refactor	none	C/PASS	C	C/PASS	–
N04	safe refactor	none	C/PASS	C	C/PASS	–
N05	safe refactor	none	C/PASS	C	C/PASS	–
N06	safe refactor (contract)	none	A/INCOMPLETE	A	A/INCOMPLETE	A/INCOMPLETE
N07	safe refactor (contract)	none	C/PASS	C	C/PASS	C/PASS
U01	unsupported: loop	defect	A/INCOMPLETE	A	A/INCOMPLETE	–
U02	unsupported: nested def	defect	C/PASS	C	A/INCOMPLETE	–
U03	unsupported: loop	none	A/INCOMPLETE	A	A/INCOMPLETE	–
U04	unsupported: ext. call	defect	A/INCOMPLETE	A	A/INCOMPLETE	–
P01	pre-existing (static)	none	A/INCOMPLETE	A	A/INCOMPLETE	–
P02	pre-existing (contract)	none	A/INCOMPLETE	A	A/INCOMPLETE	A/INCOMPLETE

Table 8: Static `guard verify` latency on synthetic repositories (ms; fresh child process per run, interpreter start included). *leaf*: one function body changes; *hub*: a constant read by most functions changes. Default 2 s quick budget. *load*: median 1-minute load average sampled after each run (10 logical CPUs; values far above 10 mean heavy contention from other processes on the host).

fns	change	mode	cache	n	wall p50	wall p95	engine p50	RSS MB	affected	verdict (runs)	load
101	leaf	quick	cold	30	493	517	156	39	2	PASS	7
101	leaf	quick	warm	30	496	598	155	40	2	PASS	6
101	leaf	full	cold	30	1,456	1,607	932	82	2	PASS	5
101	leaf	full	warm	30	668	690	167	46	2	PASS	4
101	hub	quick	cold	30	855	916	439	66	48	PASS	3
101	hub	quick	warm	30	563	604	159	42	48	PASS	3
101	hub	full	cold	30	1,450	1,536	927	81	48	PASS	3
101	hub	full	warm	30	672	704	168	45	48	PASS	3
1,001	leaf	quick	cold	30	1,445	1,488	1,016	84	2	PASS	2
1,001	leaf	quick	warm	30	1,435	1,495	1,011	84	2	PASS	2
1,001	leaf	full	cold	30	9,334	13,711	7,046	180	2	PASS	4
1,001	leaf	full	warm	30	3,388	3,599	1,197	152	2	PASS	2
1,001	hub	quick	cold	30	4,483	4,711	3,145	135	570	INCOMPLETE	4
1,001	hub	quick	warm	30	2,401	2,741	1,116	118	570	PASS	5
1,001	hub	full	cold	30	9,930	15,753	7,671	173	570	PASS	5
1,001	hub	full	warm	30	3,578	4,293	1,260	152	570	PASS	7
10,001	leaf	quick	cold	5	14,335	17,925	13,223	380	2	PASS	6
10,001	leaf	quick	warm	5	13,961	14,307	12,851	375	2	PASS	6
10,001	leaf	full	cold	5	116,354	124,784	91,375	549	2	INCOMPLETE (2); PASS (3)	6
10,001	leaf	full	warm	5	42,714	54,785	18,945	599	2	PASS	5
10,001	hub	quick	cold	5	39,896	45,567	27,315	492	5,749	INCOMPLETE	6
10,001	hub	quick	warm	5	32,328	33,837	18,679	514	5,749	INCOMPLETE	6
10,001	hub	full	cold	5	129,902	135,368	107,396	565	5,749	PASS	8
10,001	hub	full	warm	5	35,696	42,447	17,783	698	5,749	PASS	5

Table 9: Where cold `guard verify` time goes (p50 per phase, ms). *capture*: reading and hashing the candidate snapshot (working tree); *base capture*: the base commit (Git object reads); *analysis*: lowering and static analysis of the candidate (solver time included); *solver*: Z3 time alone; *impact*: matching symbols and computing the slice. *engine* is the sum of the report's timing fields excluding solver; *wall* adds interpreter start, imports and report serialization/validation. No model is involved in any of these runs.

fns	change	mode	capture	base capture	analysis	base analysis	solver	impact	engine	wall
101	leaf	quick	31	29	46	47	0	3	156	493
101	leaf	full	31	29	421	443	91	3	932	1,456
101	hub	quick	31	29	181	190	26	3	439	855
101	hub	full	31	29	425	437	91	3	927	1,450
1,001	leaf	quick	35	34	487	429	0	31	1,016	1,445
1,001	leaf	full	34	36	3,488	3,434	659	30	7,046	9,334
1,001	hub	quick	35	34	1,366	1,665	136	31	3,145	4,483
1,001	hub	full	37	38	3,704	3,803	699	32	7,671	9,930
10,001	leaf	quick	75	116	7,329	5,476	0	337	13,223	14,335
10,001	leaf	full	115	133	41,194	43,222	7,000	357	91,375	116,354
10,001	hub	quick	117	151	13,227	13,508	0	605	27,315	39,896
10,001	hub	full	129	152	56,804	50,899	8,813	367	107,396	129,902

Table 10: The two latency runs (DECISIONS I-111): the first run (pipeline order, with host load spikes) and the pre-registered repeat at the end of the pipeline, which the main text reports. Wall p50 in ms; *load*: median 1-minute load average after each run (10 logical CPUs); *ratio*: first / repeat. INC = INCOMPLETE: required obligations stayed UNKNOWN (in quick mode the 2 s budget ran out; in full mode 1–5 obligations stayed UNKNOWN without the 600 s budget being reached).

fns	change	mode	cache	n	p50 first	load	verdict first	p50 repeat	load	verdict repeat	ratio
101	leaf	quick	cold	30	498	7	PASS	493	7	PASS	1.01
101	leaf	quick	warm	30	486	6	PASS	496	6	PASS	0.98
101	leaf	full	cold	30	1,653	6	PASS	1,456	5	PASS	1.14
101	leaf	full	warm	30	701	6	PASS	668	4	PASS	1.05
101	hub	quick	cold	30	911	6	PASS	855	3	PASS	1.06
101	hub	quick	warm	30	636	7	PASS	563	3	PASS	1.13
101	hub	full	cold	30	1,645	6	PASS	1,450	3	PASS	1.13
101	hub	full	warm	30	824	9	PASS	672	3	PASS	1.22
1,001	leaf	quick	cold	30	1,498	11	PASS	1,445	2	PASS	1.04
1,001	leaf	quick	warm	30	1,478	9	PASS	1,435	2	PASS	1.03
1,001	leaf	full	cold	30	10,048	5	PASS	9,334	4	PASS	1.08
1,001	leaf	full	warm	30	3,655	9	PASS	3,388	2	PASS	1.08
1,001	hub	quick	cold	30	4,526	7	INC	4,483	4	INC	1.01
1,001	hub	quick	warm	30	2,745	21	PASS	2,401	5	PASS	1.14
1,001	hub	full	cold	30	27,684	36	INC (7); PASS (23)	9,930	5	PASS	2.79
1,001	hub	full	warm	30	11,378	29	PASS	3,578	7	PASS	3.18
10,001	leaf	quick	cold	5	59,690	31	PASS	14,335	6	PASS	4.16
10,001	leaf	quick	warm	5	16,179	20	PASS	13,961	6	PASS	1.16
10,001	leaf	full	cold	5	133,850	8	INC (1); PASS (4)	116,354	6	INC (2); PASS (3)	1.15
10,001	leaf	full	warm	5	38,258	5	PASS	42,714	5	PASS	0.90
10,001	hub	quick	cold	5	34,733	4	INC	39,896	6	INC	0.87
10,001	hub	quick	warm	5	33,342	8	INC	32,328	6	INC	1.03
10,001	hub	full	cold	5	118,210	6	PASS	129,902	8	PASS	0.91
10,001	hub	full	warm	5	34,941	8	PASS	35,696	5	PASS	0.98